

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №1

з дисципліни
«Об'єктно-орієнтоване програмування»
на тему

Знайомство із середовищем розробки та складання модульних проєктів програм.

Виконав:

студент групи ІМ-55

Матвіїв Ростислав Степанович

Номер в списку - 15

Перевірив:

Рекечинський Д. О.

Завдання:

1. Створити у обраному середовищі розробки проєкт з ім'ям Lab1.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту:

B1: 3

Вікно діалогу з елементом списку (List Box) та дві кнопки: [Так] і [Відміна]. У список автоматично записуються назви груп факультету. Після вибору рядка списку і натискування кнопки [Так] вибраний текст буде відображатися у головному вікні.

B2: 0

Вікно діалогу для вводу тексту, яке має стрічку вводу (Edit Control) та дві кнопки: [Так] і [Відміна]. Якщо ввести рядок тексту і натиснути [Так], то у головному вікні повинен відображатися текст, що був введений.

Робота:

Lab1.cpp

```
#include <windows.h>
#include "resource.h"
#include "module1.h"
#include "module2.h"
```

```
namespace {
```

```

const wchar_t kWindowClassName[] = L"Lab1MainWindowClass";
const wchar_t kWindowTitle[] = L"Лабораторна робота №1 (Ж=15, IM-55)";

HINSTANCE g_hInstance = nullptr;
wchar_t g_displayText[256] = L"";

void OnPaint(HWND hWnd) {
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);
    RECT rc;
    GetClientRect(hWnd, &rc);
    rc.left += 20; rc.top += 20;
    DrawTextW(hdc, g_displayText, -1, &rc, DT_LEFT | DT_TOP | DT_WORDBREAK);
    EndPaint(hWnd, &ps);
}

LRESULT CALLBACK MainWndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {
    switch (message) {
        case WM_COMMAND:
            switch (LOWORD(wParam)) {
                case ID_ROBOTA_ROBOTA1: {
                    wchar_t buffer[256] = L"";
                    if (ShowRobota1Dialog(hWnd, g_hInstance, buffer, 256) == IDOK) {
                        lstrcpynW(g_displayText, buffer, 256);
                        InvalidateRect(hWnd, nullptr, TRUE);
                    }
                    return 0;
                }
                case ID_ROBOTA_ROBOTA2: {
                    wchar_t buffer[256] = L"";
                    if (ShowRobota2Dialog(hWnd, g_hInstance, buffer, 256) == IDOK) {
                        lstrcpynW(g_displayText, buffer, 256);
                        InvalidateRect(hWnd, nullptr, TRUE);
                    }
                    return 0;
                }
            }
            break;
        case WM_PAINT:
            OnPaint(hWnd);
            return 0;
        case WM_DESTROY:
            PostQuitMessage(0);
            return 0;
    }
    return DefWindowProcW(hWnd, message, wParam, lParam);
}

ATOM RegisterMainWindowClass(HINSTANCE hInstance) {

```

```

WNDCLASSEXW wc = {};
wc.cbSize = sizeof(wc);
wc.style = CS_HREDRAW | CS_VREDRAW;
wc.lpfnWndProc = MainWndProc;
wc.hInstance = hInstance;
wc.hIcon = LoadIconW(nullptr, IDI_APPLICATION);
wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_WINDOW + 1);
wc.lpszMenuName = MAKEINTRESOURCEW(IDR_MAINMENU);
wc.lpszClassName = kWindowClassName;
wc.hIconSm = LoadIconW(nullptr, IDI_APPLICATION);
return RegisterClassExW(&wc);
}

}

int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE, PWSTR, int nCmdShow) {
    g_hInstance = hInstance;
    if (!RegisterMainWindowClass(hInstance)) {
        MessageBoxW(nullptr, L"Не вдалося зареєструвати клас вікна.", L"Помилка",
MB_ICONERROR);
        return 1;
    }
    HWND hWnd = CreateWindowExW(0, kWindowClassName, kWindowTitle,
WS_OVERLAPPEDWINDOW,
    CW_USEDEFAULT, CW_USEDEFAULT, 520, 360, nullptr, nullptr, hInstance,
nullptr);
    if (!hWnd) {
        MessageBoxW(nullptr, L"Не вдалося створити вікно.", L"Помилка",
MB_ICONERROR);
        return 1;
    }
    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);
    MSG msg;
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }
    return static_cast<int>(msg.wParam);
}

```

module1.cpp

```

#include "module1.h"
#include "resource.h"

```

```

namespace {

const wchar_t* const kGroups[] = {
    L"IM-55", L"IP-51", L"IA-53", L"IC-52"
};
const int kGroupsCount = static_cast<int>(sizeof(kGroups) / sizeof(kGroups[0]));

struct DialogParams {
    wchar_t* buffer;
    int bufferSize;
};

INT_PTR CALLBACK Robota1DialogProc(HWND hDlg, UINT message, WPARAM wParam, LPARAM
lParam) {
    switch (message) {
        case WM_INITDIALOG: {
            SetWindowLongPtrW(hDlg, DWLP_USER, static_cast<LONG_PTR>(lParam));
            HWND hList = GetDlgItem(hDlg, IDC_LIST_GROUPS);
            for (int i = 0; i < kGroupsCount; ++i)
                SendDlgItemMessageW(hList, LB_ADDSTRING, 0,
reinterpret_cast<LPARAM>(kGroups[i]));
            SendDlgItemMessageW(hList, LB_SETCURSEL, 0, 0);
            return TRUE;
        }
        case WM_COMMAND:
            switch (LOWORD(wParam)) {
                case IDOK: {
                    auto* params =
reinterpret_cast<DialogParams*>(GetWindowLongPtrW(hDlg, DWLP_USER));
                    HWND hList = GetDlgItem(hDlg, IDC_LIST_GROUPS);
                    int sel = static_cast<int>(SendDlgItemMessageW(hList, LB_GETCURSEL, 0, 0));
                    if (sel != LB_ERR && params && params->buffer && params->bufferSize >
0) {
                        int len = static_cast<int>(SendDlgItemMessageW(hList, LB_GETTEXTLEN,
sel, 0));
                        if (len >= 0 && len < params->bufferSize)
                            SendDlgItemMessageW(hList, LB_GETTEXT, sel,
reinterpret_cast<LPARAM>(params->buffer));
                        else
                            params->buffer[0] = L'\0';
                    }
                    EndDialog(hDlg, IDOK);
                    return TRUE;
                }
                case IDCANCEL:
                    EndDialog(hDlg, IDCANCEL);
                    return TRUE;
            }
            break;
    }
    return FALSE;
}

```

```

}

}

int ShowRobota1Dialog(HWND hParentWnd, HINSTANCE hInstance,
                      wchar_t* outBuffer, int outBufferSize) {
    DialogParams params{ outBuffer, outBufferSize };
    return static_cast<int>(DialogBoxParamW(
        hInstance, MAKEINTRESOURCEW(IDD_DLG_ROBOTA1), hParentWnd,
        Robota1DialogProc, reinterpret_cast<LPARAM>(&params)));
}

```

module1.h

```

#pragma once

#include <windows.h>

int ShowRobota1Dialog(HWND hParentWnd, HINSTANCE hInstance,
                      wchar_t* outBuffer, int outBufferSize);

```

m

odule2.cpp

```

#include "module2.h"
#include "resource.h"

namespace {

struct DialogParams {
    wchar_t* buffer;
    int bufferSize;
};

INT_PTR CALLBACK Robota2DialogProc(HWND hDlg, UINT message, WPARAM wParam, LPARAM
lParam) {
    switch (message) {
    case WM_INITDIALOG:
        SetWindowLongPtrW(hDlg, DWLP_USER, static_cast<LONG_PTR>(lParam));
        return TRUE;
    }
}

```

```

        case WM_COMMAND:
            switch (LOWORD(wParam)) {
                case IDOK: {
                    auto* params =
reinterpret_cast<DialogParams*>(GetWindowLongPtrW(hDlg, DWLP_USER));
                    if (params && params->buffer && params->bufferSize > 0)
                        GetDlgItemTextW(hDlg, IDC_EDIT_TEXT, params->buffer, params-
>bufferSize);
                    EndDialog(hDlg, IDOK);
                    return TRUE;
                }
                case IDCANCEL:
                    EndDialog(hDlg, IDCANCEL);
                    return TRUE;
            }
            break;
    }
    return FALSE;
}

}

int ShowRobota2Dialog(HWND hParentWnd, HINSTANCE hInstance,
                    wchar_t* outBuffer, int outBufferSize) {
    DialogParams params{ outBuffer, outBufferSize };
    return static_cast<int>(DialogBoxParamW(
        hInstance, MAKEINTRESOURCEW(IDD_DLG_ROBOTA2), hParentWnd,
        Robota2DialogProc, reinterpret_cast<LPARAM>(&params)));
}

```

module2.h

```

#pragma once
#include <windows.h>

int ShowRobota2Dialog(HWND hParentWnd, HINSTANCE hInstance,
                    wchar_t* outBuffer, int outBufferSize);

```

resource.h

```
#pragma once
```

```
#define IDR_MAINMENU          101
#define ID_ROBOTA_ROBOTA1    40001
#define ID_ROBOTA_ROBOTA2    40002

#define IDD_DLG_ROBOTA1      201
#define IDD_DLG_ROBOTA2      202

#define IDC_LIST_GROUPS      1001
#define IDC_EDIT_TEXT        1002
```

resource.rc

```
#pragma code_page(65001)    // говорит windres, что строки в UTF-8
```

```
#include <windows.h>
```

```
#include "resource.h"
```

```
IDR_MAINMENU MENU
```

```
BEGIN
```

```
    POPUP "Робота"
```

```
    BEGIN
```

```
        MENUITEM "Робота1", ID_ROBOTA_ROBOTA1
```

```
        MENUITEM "Робота2", ID_ROBOTA_ROBOTA2
```

```
    END
```

```
END
```

```
IDD_DLG_ROBOTA1 DIALOGEX 0, 0, 200, 130
```

```
STYLE DS_SETFONT | DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU
```

```
CAPTION "Робота1 – Группы факультету"
```

```
FONT 9, "MS Shell Dlg"
```

```
BEGIN
```

```
        LISTBOX            IDC_LIST_GROUPS, 10, 10, 180, 85, LBS_NOTIFY |  
LBS_STANDARD | WS_TABSTOP
```

```
        DEFPUSHBUTTON      "Так",          IDOK,          45, 105, 50, 14
```

```
        PUSHBUTTON         "Відміна",      IDCANCEL, 105, 105, 50, 14
```

```
END
```

```
IDD_DLG_ROBOTA2 DIALOGEX 0, 0, 200, 70
```

```
STYLE DS_SETFONT | DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU
```

```
CAPTION "Робота2 – Введення тексту"
```

```
FONT 9, "MS Shell Dlg"
```

```
BEGIN
```

```
        EDITTEXT           IDC_EDIT_TEXT, 10, 15, 180, 14, ES_AUTOHSCROLL |  
WS_TABSTOP | WS_BORDER
```

```
        DEFPUSHBUTTON      "Так",          IDOK,          45, 45, 50, 14
```

```
        PUSHBUTTON         "Відміна",      IDCANCEL, 105, 45, 50, 14
```

```
END
```

Makefiles

```
CXX      := g++
```

```
WINDRES  := windres
```

```
SRC_DIR  := src
```

```
RES_DIR  := res
```

```
BUILD_DIR := build
```

```
CXXFLAGS := -std=c++17 -municode -DUNICODE -D_UNICODE -Wall -Wextra -I$(SRC_DIR)
```

```
LDLFLAGS := -municode -mwindows
```

```
LDLIBS   := -luser32 -lgdi32 -lcomdlg32
```

```
TARGET := Lab1.exe
```

```
SOURCES := $(SRC_DIR)/Lab1.cpp $(SRC_DIR)/module1.cpp $(SRC_DIR)/module2.cpp
```

```
OBJECTS := $(subst $(SRC_DIR)/%.cpp,$(BUILD_DIR)/%.o,$(SOURCES))
```

```
RES_OBJ := $(BUILD_DIR)/resource.res.o

.PHONY: all clean

all: $(TARGET)

$(TARGET): $(OBJECTS) $(RES_OBJ)
    $(CXX) $(OBJECTS) $(RES_OBJ) -o $@ $(LDFLAGS) $(LDLIBS)

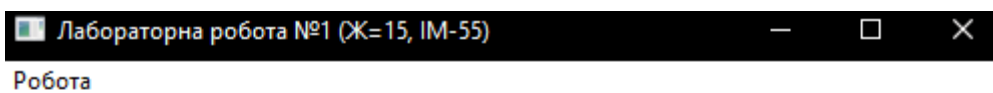
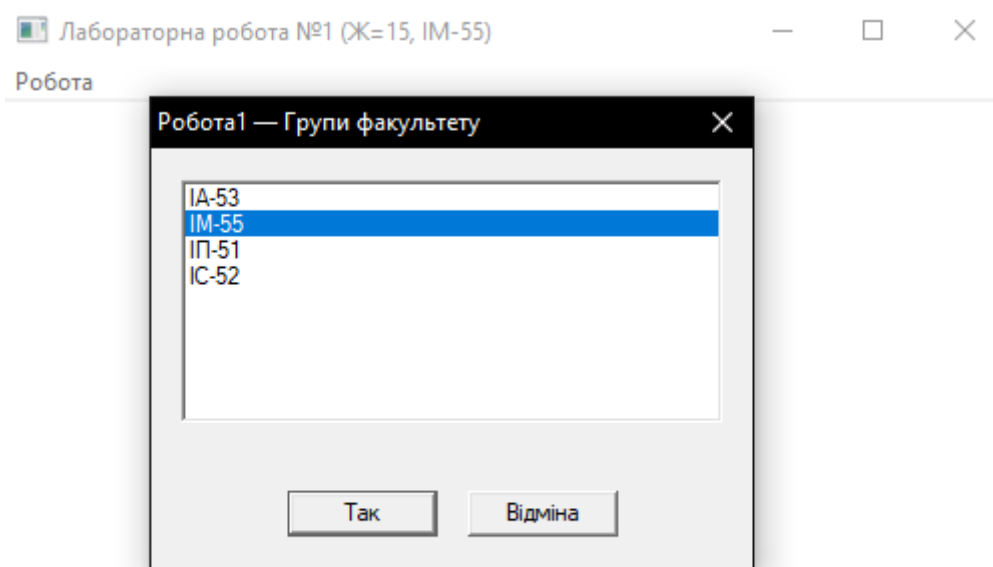
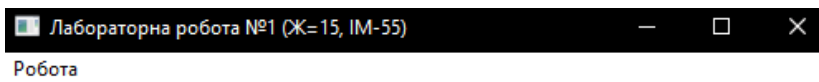
$(BUILD_DIR)/%.o: $(SRC_DIR)/%.cpp | $(BUILD_DIR)
    $(CXX) $(CXXFLAGS) -c $< -o $@

$(RES_OBJ): $(RES_DIR)/resource.rc $(SRC_DIR)/resource.h | $(BUILD_DIR)
    $(WINDRES) -I$(SRC_DIR) $< -O coff -o $@

$(BUILD_DIR):
    mkdir -p $(BUILD_DIR)

clean:
    rm -rf $(BUILD_DIR) $(TARGET)
```

Скріншоти

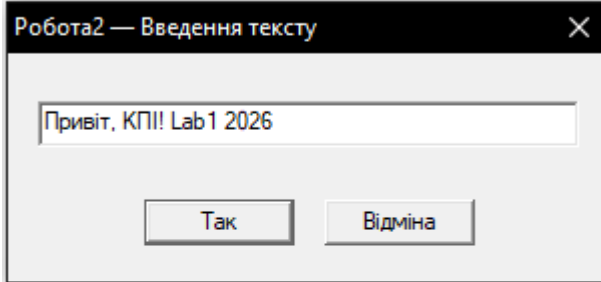


ІМ-55

Лабораторна робота №1 (Ж=15, ІМ-55)



Робота



Лабораторна робота №1 (Ж=15, ІМ-55)



Робота

Привіт, КПП! Lab1 2026

Висновки

У ході роботи отримано навички створення програм для Windows із використанням Win32 API: реєстрація та створення головного вікна, цикл обробки повідомлень, віконна процедура, меню, модальні діалогові вікна з ресурсів. Логіку B1 і B2 реалізовано у двох незалежних модулях (module1, module2); назвні модулі відкривають лише одну функцію виклику діалогу, а callback-функції діалогів приховано в анонімному просторі імен. Ресурси (меню та діалоги) описано в єдиному файлі resource.rc. Збирання виконується за допомогою Makefile (g++, windres), текст обробляється в Unicode (wchar_t), тому кирилиця відображається коректно. Перевірка показала, що програма працює відповідно до завдання.